

DECODELABS DATA SCIENCE INTERNSHIP

# Project 1 — Advanced EDA & Feature Engineering

Data Cleaning, Feature Engineering & Reproducible E-commerce Analytics

|                              |                                    |                                 |
|------------------------------|------------------------------------|---------------------------------|
| <b>PROGRAM</b><br>Batch 2026 | <b>ROLE</b><br>Data Science Intern | <b>AUTHOR</b><br>Muhammad Usaid |
|------------------------------|------------------------------------|---------------------------------|

Internship Project Documentation

# 1 OBJECTIVE

Build an end-to-end, production-style data cleaning and feature engineering pipeline for a 1,200-row e-commerce orders dataset. The project emphasizes mathematical clarity before modeling through statistical imputation, IQR-based outlier treatment, engineered features, encoding, scaling, schema validation, and reproducible testing.

## AT A GLANCE

The pipeline keeps all 1,200 rows, removes all missing values, neutralizes detected outliers without deleting records, creates seven engineered features, standardizes eight continuous variables, and validates the final schema. The repository contains 17 unit tests covering the pipeline stages.

# 2 METHODOLOGY

## 2.1 DATASET

The source is an e-commerce dataset containing 1,200 order line items and 14 columns, including OrderID, Date, CustomerID, Product, Quantity, UnitPrice, PaymentMethod, OrderStatus, ItemsInCart, CouponCode, ReferralSource, and TotalPrice.

## 2.2 MISSING DATA HANDLING

| Missingness | Strategy                                                                                      |
|-------------|-----------------------------------------------------------------------------------------------|
| < 5%        | Drop rows                                                                                     |
| 5–20%       | Statistical imputation — median for skewed numeric data or subgroup mode for categorical data |
| > 20%       | KNN-based multi-dimensional estimation                                                        |

CouponCode had 309 missing values (25.75%). Rather than inventing a coupon code through KNN, the pipeline first preserves the missingness signal as CouponUsed and then fills the categorical field with the domain-meaningful sentinel “NoCoupon”.

## 2.3 OUTLIER DETECTION & NEUTRALIZATION

Numeric columns are checked using IQR fences: Lower Bound =  $Q1 - 1.5 \times IQR$  and Upper Bound =  $Q3 + 1.5 \times IQR$ . Only TotalPrice contains genuine outliers in the documented run: 8 rows (0.67%). These values are winsorized with `numpy.clip` rather than deleted, and an outlier flag preserves the information that the value was extreme.

## 2.4 FEATURE ENGINEERING

| Feature            | Formula / Source        | Purpose                            |
|--------------------|-------------------------|------------------------------------|
| OrderMonth         | Date.month              | Captures monthly seasonality       |
| OrderDayOfWeek     | Date.dayofweek          | Captures weekday purchase patterns |
| IsWeekendOrder     | Day $\in$ {5, 6}        | Weekend indicator                  |
| CouponUsed         | CouponCode.notna()      | Preserves informative missingness  |
| CartFillRatio      | Quantity / ItemsInCart  | Proxy for cart utilization         |
| CustomerOrderCount | CustomerID value counts | Customer frequency                 |
| IsRepeatCustomer   | CustomerOrderCount > 1  | Repeat-customer indicator          |

## 2.5 CATEGORICAL ENCODING & MULTICOLLINEARITY

Nominal variables are one-hot encoded to avoid introducing false ordinal relationships. The numeric feature matrix is checked for multicollinearity using an absolute-correlation threshold of 0.80. No multicollinear feature pairs were found in the documented run.

## 2.6 FEATURE SCALING

Continuous numeric features are standardized with scikit-learn StandardScaler after winsorization. Eight continuous features are scaled; binary indicators and one-hot encoded columns remain untouched.

## 2.7 SCHEMA VALIDATION

A dependency-free runtime schema contract checks expected dtypes, nullability, and value bounds. Validation is lazy, collecting all detected failures into a single report instead of stopping at the first error.

## 3 RESULTS

| Metric                        | Before           | After                   |
|-------------------------------|------------------|-------------------------|
| Rows                          | 1,200            | 1,200 (no rows dropped) |
| Columns                       | 14               | 25 (+11)                |
| Missing values                | 309 (CouponCode) | 0                       |
| Outliers                      | 8 (TotalPrice)   | 0 (winsorized, flagged) |
| Multicollinear pairs          | —                | 0                       |
| Numeric features standardized | —                | 8                       |
| Schema validation             | —                | Passed                  |

### 3.1 BUSINESS INSIGHTS

- The seven products are broadly balanced, with each representing approximately 13.0%–15.1% of orders.
- Payment methods are also relatively evenly distributed.
- Cancelled and Returned orders together are more common than Delivered orders in the documented run.
- 74.2% of orders used a coupon code.
- 29.8% of orders occurred on weekends.
- Only 1.8% of customers placed more than one order in this sample.
- Average order value is \$1,053.64 and average cart-fill ratio is 0.58.

### 3.2 REPRODUCIBILITY

The repository provides a single `main.py` entry point that regenerates the processed datasets and reports. The project also includes a pre-executed Jupyter notebook and a pytest suite.

## 4 TECHNICAL IMPLEMENTATION

| Area                     | Technology / Component      |
|--------------------------|-----------------------------|
| Language                 | Python 3.12                 |
| Data manipulation        | pandas, NumPy               |
| Imputation               | scikit-learn KNNImputer     |
| Scaling                  | scikit-learn StandardScaler |
| Visualization            | matplotlib, seaborn         |
| Testing                  | pytest                      |
| Documentation / analysis | Jupyter / nbconvert         |

## 5 TESTING

The project contains 17 unit tests covering missing-value routing, IQR outlier detection, winsorization without row loss, engineered features, one-hot encoding, multicollinearity detection, StandardScaler behavior, and schema validation.

## 6 KEY TAKEAWAYS

- Data cleaning is treated as an engineering pipeline rather than a manual preprocessing step.
- Domain knowledge can justify overriding a generic statistical rule when missingness itself carries information.
- Winsorization can preserve records while controlling the influence of extreme values.
- Feature engineering converts raw transactional columns into more useful analytical signals.
- Schema validation and automated tests make the workflow reproducible and auditable.

**Author: Muhammad Usaid — DecodeLabs Data Science Intern, Batch 2026**

Source: DecodeLabs Internship Project 1 repository and README.